FraudArena

Phase 1: Foundation (H0 to H4)

Hour-by-hour build sheet for the first four hours

COLOSSUS 3.0 FINALS · 13 TO 14 NOVEMBER 2026

13 November 2026 · Gate at H4: one transaction in, one decision out, from the API

Numbers marked "target" or "illustrative" are to be replaced by values measured in our own runs.

1. Phase 1 in one page

Goal: by H4 the simulated bank runs end to end: a transaction goes in through the API, features are computed, rules are applied, a decision (allow, hold or block) comes out and is stored. Nothing smart yet. No attacker, no ML, no LLM. Everything later stands on this layer, so it must be boring and correct.

Deliverable	Done when
Repo and Compose stack	`docker compose up` starts api, db and web; one command, fresh clone works
Contracts frozen	Event, State, Rule, Decision types in `fa/contracts.py`; nobody edits them after H1 without telling everyone
Simulator core	simulated clock, accounts, payees, devices, `apply()`, snapshot and restore
Features	`compute_features(state, event)` returns the 12 agreed features, point in time only
Rule engine	JSON rules, precedence block > hold > allow, baseline rule set v1
Storage and API	`POST /tx`, `GET /decisions`, `GET /health`; rows land in Postgres
Tests	10 unit tests pass in CI-style one command: `make test`

Gate at H4: run `make demo-tx`. It must show a normal payment allowed, a large payment to a brand-new payee held, and a rapid fan-out blocked, all read back from the database. If this does not work at H4, stop everything else and fix it.

2. Hour by hour

Time	Work	Output
H0:00 to H0:30	Agree this plan, create repo, pick who owns which folder (section 7), set up Python 3.11 venv, Node, Docker. Push empty skeleton.	repo with folders, README, Makefile
H0:30 to H1:00	Write and freeze contracts (section 4). Write `params.yaml` with every threshold and seed.	contracts.py, params.yaml
H1:00 to H2:00	Simulator and features (section 5), rule engine (section 6), in parallel with Compose, Postgres schema and FastAPI skeleton (section 8).	sim runs in a plain script; api returns /health
H2:00 to H3:00	Wire api to the engine and database. Write the normal-traffic generator (200 accounts, 7 days). Write baseline rules v1.	POST /tx returns a decision and stores it
H3:00 to H3:45	Unit tests (section 9), fix bugs, `make demo-tx`.	10 tests green
H3:45 to H4:00	Gate check, commit, tag `phase1-done`, hand over to Phase 2 (attacker).	git tag

3. Repo layout

```
fraudarena/
  params.yaml            every threshold, seed, size in one place
  Makefile               up, down, test, demo-tx, lint
  docker-compose.yml     api, db, web
  fa/
    contracts.py         Event, State, Rule, Decision (frozen at H1)
    core/
      clock.py           simulated clock
      state.py           accounts, payees, devices, snapshot/restore
      features.py        compute_features(state, event)
      engine.py          apply(state, event); decide(features, rules)
    world/normal.py      normal-traffic generator
    rules/baseline_v1.json
  api/main.py            FastAPI app
  db/schema.sql          tables
  tests/                 pytest
  web/                   React + Vite (starts empty in Phase 1)
```

4. Contracts to freeze at H1

Four small types. Everything else in the project passes these around, so changing them later breaks every module.

```
Event:     id, ts (sim seconds), account, type, amount,
           payee, device, channel
           type in {login, device_add, payee_add, pay, cashout}
State:     per-account balance, avg_30d, payees{id: first_seen_ts},
           devices{id: first_seen_ts}, recent_pays[(ts, amt, payee)]
Rule:      {id, action: hold|block,
            when: [{feature, op, value}, ...]}      # AND of conditions
           op in {<, <=, >, >=, ==}; at most 4 conditions
Decision:  {event_id, action: allow|hold|block,
            rule_id|None, version, features}
```

Rule format is the key choice. Rules are data, not code. This is what lets the defender propose patches later and the verifier replay them safely. Do not use free-form Python expressions or LLM-written code as rules.

5. Simulator and features

- **Simulated clock:** time is an integer of seconds that only moves when the script advances it. Never call `time.time()` inside the engine. This makes every run repeatable and lets a week of traffic replay in seconds.
- **Seed everything:** one `random.Random(seed)` passed in; the seed comes from `params.yaml`.
- **apply(state, event):** the only function that changes state (balance, payee list, device list, recent payments). Both the live engine and the later ML export call it, so training data and live behaviour cannot drift apart.
- **snapshot() and restore():** deep copy of state at a point in time. Later phases reset to a snapshot thousands of times during attacker search, so keep state in plain dicts and lists and make the copy cheap.
- **Normal traffic:** 200 accounts, 7 days, mostly small pays to a few regular payees, a few unusual but legitimate events (rent to a new landlord, new phone). This is also the base for the friction and golden-benign checks.

The 12 features (computed before applying the event, from state only, never from the future):

Feature	Meaning
amount_ratio_30d	amount divided by the account 30-day average payment
payee_age_h	hours since this payee was first added (0 if new)
device_changed_24h	1 if a new device was added in the last 24 hours
n_pay_1h, n_pay_24h	number of payments in the last 1 hour and 24 hours
sum_1h, sum_24h	total paid in the last 1 hour and 24 hours
sum_newpayee_1h	total sent to payees younger than 24 hours in the last hour
fan_out_1h	distinct payees paid in the last hour
r_fan_in_24h, r_passthrough_1h	receiving side: distinct senders in 24 hours; share of received money sent onward within the hour
hour_of_day, night_flag	hour 0 to 23; 1 if between 00:00 and 05:00

6. Rule engine and baseline rules

- **decide(features, rules):** evaluate every rule. A rule fires when all its conditions are true. If any block rule fires, the decision is block; else if any hold rule fires, hold; else allow. Return the id of the first rule that decided.
- **Versions:** a rule set is a list of rules plus a version number. Store each version as a row; never edit one in place. Phase 4 release depends on this.
- **Baseline v1:** deliberately simple and typical of a bank, so the attacker has something to beat in Phase 2. Aim for 5 or 6 rules.

```
R1 hold  if amount_ratio_30d >= 10
R2 hold  if payee_age_h < 1 and amount_ratio_30d >= 3
R3 block if n_pay_1h >= 12
R4 hold  if device_changed_24h == 1 and sum_1h >= 50000
R5 block if fan_out_1h >= 8
R6 hold  if night_flag == 1 and amount_ratio_30d >= 6
```

These thresholds are placeholders. Check them against the generated normal traffic so that the hold rate on normal traffic is under 1%, and note the measured value in the README.

7. Division of work: M1 to M4

Four people, four folders, no shared files except the frozen contracts. M1 to M4 are fixed labels; assign them to people at H0:00 and keep them for all 24 hours.

Member	Role	Owns (only they edit)	Phase 1 output
M1	Lead and integrator	fa/contracts.py, params.yaml, Makefile, README, tests/test_api.py	frozen contracts, params, final gate run
M2	Arena: API and infrastructure	api/, db/, docker-compose.yml, web/ skeleton	Compose stack, Postgres, /tx /decisions /health
M3	Data and ML: world and features	fa/core/clock.py, state.py, features.py, fa/world/	simulator, 12 features, normal traffic
M4	Verifier and release: rules	fa/core/engine.py, fa/rules/, tests/ (except test_api)	decide(), rule loader, baseline v1, 10 tests

M1: Lead and integrator

Time	Task	Done when
H0:00 to H0:30	Create repo, folders, Makefile, README; add all four to the repo; assign M1 to M4	everyone can push
H0:30 to H1:00	Write and freeze contracts.py (section 4) and params.yaml (seed, 200 accounts, 7 days, all thresholds); announce freeze	contracts merged, team told
H1:00 to H3:00	Review every merge, resolve interface mismatches, keep README commands current, write make targets (up, test, demo-tx)	make targets run
H3:00 to H3:45	Write tests/test_api.py (test 10); run full gate script with the team	make demo-tx prints allow, hold, block
H3:45 to H4:00	Tag phase1-done; post make up, make test, make demo-tx in team chat; baseline hold rate and thresholds in README	tag pushed

M2: Arena, API and infrastructure

Time	Task	Done when
H0:30 to H1:30	docker-compose.yml with db (postgres:16), api, web placeholder; db/schema.sql (section 8)	docker compose up starts all three
H1:30 to H2:30	FastAPI app: /health, load baseline_v1.json into rule_versions on start; stub /tx that returns allow	GET /health returns ok and version 1
H2:30 to H3:15	Wire /tx to M3 state and features and M4 decide(); write the row into events; implement /decisions	POST /tx returns a real decision and a row appears in Postgres
H3:15 to H4:00	Fix integration bugs found in the gate run; confirm a fresh clone starts with one command	fresh clone works

Fallback: if Postgres or Compose fails at H1:30, use SQLite and run the api directly; keep the same SQL.

M3: Data and ML, simulator and features

Time	Task	Done when
H0:30 to H1:00	Read contracts draft; sketch state structure (balance, avg_30d, payees, devices, recent_pays)	agreed with M1
H1:00 to H2:00	clock.py, state.py (snapshot, restore), apply(state, event) in engine-side helper, features.py with all 12 features	tests 2 to 6 pass on a hand-built example
H2:00 to H3:00	world/normal.py: 200 accounts, 7 days, seeded; include a few unusual but legitimate cases (rent to new landlord, new phone)	same seed gives identical traffic (test 1)
H3:00 to H3:45	Run normal traffic through M4 decide(); measure hold+block rate; help M4 tune baseline thresholds so it is under 1% (test 9)	measured rate written in README
H3:45 to H4:00	Export a 1-page note: feature definitions as implemented	note in README

Warning: features must use state before the current event is applied. This is the leakage rule; test 3 enforces it.

M4: Verifier and release, rules and tests

Time	Task	Done when
H0:30 to H1:00	Read the Rule format in contracts; write the JSON schema for rules (allowed features, ops, max 4 conditions)	schema file in rules/
H1:00 to H2:00	engine.py decide(features, rules) with precedence block > hold > allow; rules loader that rejects unknown features or operators	tests 7 and 8 pass
H2:00 to H3:00	rules/baseline_v1.json (R1 to R6, section 6); write unit tests 1 to 8 skeleton so M3 and M2 can run them	make test runs, even if some fail
H3:00 to H3:45	Test 9 with M3; tune thresholds; keep tests green	under 1% hold rate
H3:45 to H4:00	Write the Phase 2 note: how the attacker will call decide() and replay a plan	1 paragraph in README

Hand-offs between members

From	To	What	Needed by
M1	M2, M3, M4	contracts.py and params.yaml frozen	H1:00
M3	M4	features.py with compute_features() working	H2:00
M4	M2	decide() and baseline_v1.json	H2:30
M3	M2	state and apply() importable	H2:30
M2	M1	working /tx and /decisions for the gate run	H3:15
M3 and M4	M1	measured hold rate on normal traffic	H3:45

If a hand-off is late: the receiver writes a stub (a function that returns a fixed value) and carries on; the sender announces the real one as soon as it is merged. Nobody waits idle. Only M1 edits contracts.py. Merge rule: small commits to main every 30 to 45 minutes, pull before you push.

8. Storage, API and Compose

```
-- db/schema.sql  (Phase 1 subset)
CREATE TABLE rule_versions(
  version INT PRIMARY KEY, created_at TIMESTAMPTZ DEFAULT now(),
  rules_json JSONB NOT NULL, source TEXT);
CREATE TABLE events(
  id BIGSERIAL PRIMARY KEY, run_id TEXT, ts BIGINT, account TEXT,
  type TEXT, amount NUMERIC, payee TEXT, device TEXT,
  action TEXT, rule_id TEXT, version INT, features JSONB);
```

```
POST /tx          body: Event       -> Decision (and row in events)
GET  /decisions   ?run_id=&limit=   -> list of recent decisions
GET  /health      -> {"ok": true, "rule_version": 1}
```

- **Compose services:** db (postgres:16, volume), api (python:3.11-slim, uvicorn, depends on db), web (node, empty placeholder for now).
- Load `rules/baseline_v1.json` into `rule_versions` at API start if the table is empty.
- Write events in batches of 1,000 when the simulator runs in bulk. The single-event path (`POST /tx`) can write one row at a time.

9. The 10 tests and the H4 gate

#	Test	Why
1	same seed gives identical normal traffic twice	repeatability
2	snapshot, run 100 events, restore: state equals the snapshot	fast resets later
3	features at event t never read data from after t	no leakage
4	amount_ratio_30d correct on a hand-built example	feature maths
5	payee_age_h is 0 for a new payee and grows with sim time	feature maths
6	fan_out_1h counts distinct payees only inside the window	window logic
7	block beats hold beats allow when several rules fire	precedence
8	a rule with an unknown feature or operator is rejected by the loader	safety
9	normal traffic hold+block rate is under 1%	baseline sanity
10	POST /tx returns the same decision as calling decide() directly	api wiring

H4 gate script (`make demo-tx`) sends three events and prints the decision read back from Postgres:

```
1. normal pay, small, known payee          -> allow
2. pay 12x average to a payee added 10 min -> hold   (R2)
3. 9 payees in one hour from one account   -> block  (R5)
```

10. Risks and what to cut

Risk	Cut or fix
Compose or Postgres wastes more than 45 minutes	run the api against SQLite for now, switch back after the gate; the schema is plain SQL
Feature code disagrees between people	one file, one owner (Data and ML); everyone imports it, nobody copies it
Normal traffic triggers too many holds	raise thresholds in baseline v1 and write down the new numbers; do not hide it in the code
Contracts keep changing	lead freezes at H1; changes need all four people to agree out loud
Behind schedule at H3	drop the web placeholder and tests 9 and 10 until after the gate; never drop the gate itself